GPS C/A Code Generator

Verilog Design Verification Report — Internship Task 1

1. Objective

The task was to verify a pre-existing Verilog implementation of a GPS C/A (Coarse/Acquisition) code generator, based on the Gold code construction described in Section 2.3.3 of the reference GPS textbook. The C/A code generator produces the unique pseudo-random noise (PRN) sequence that each GPS satellite transmits, allowing a receiver to identify and separate signals coming from different satellites that all share the same frequency band.

The verification was split between two internees: one covering PRN 1–16, and the other covering PRN 17–32, each running an independent self-checking testbench against the same shared design files.

2. Background: How the C/A Code Is Built

The C/A code is a Gold code formed by combining the outputs of two 10-bit linear feedback shift registers (LFSRs), referred to as G1 and G2. Both registers use maximal-length feedback polynomials and are reset to the all-ones state at the start of every code epoch. G1's feedback taps are fixed (cells 3 and 10), while G2 uses a fixed 6-tap feedback polynomial (cells 2,3,6,8,9,10) for its internal shifting. What makes each satellite's code unique is not G1 or G2 themselves — they run identically for every satellite — but a per-satellite *code phase selector* that XORs two specific cells of the G2 register together, as defined in Table 2.3 of the reference text.

3. Design Files

File	Purpose
g1_lfsr.v	10-bit LFSR implementing the G1 generator (fixed taps: cells 3 & 10).
g2_lfsr.v	10-bit LFSR implementing the G2 generator (fixed taps: cells 2,3,6,8,9,10).
prn_selector.v	Selects two cells of the G2 register per satellite (Table 2.3) and XORs them.
gps_ca_top.v	Top-level module; XORs G1's output with the selected G2 tap to form the C/A chip.
tb_gps_ca_prn1_16.v	Self-checking testbench covering PRN 1-16 (Internee 1).
tb_gps_ca_prn17_32.v	Self-checking testbench covering PRN 17-32, plus a PRN34==PRN37 cross-check (Internee 2).

4. Issues Found and Fixes Applied

4.1 Incomplete PRN selector (prn_selector.v)

The original module only implemented case statements for PRN 1 through PRN 4. Every other PRN (5 through 32) fell through to the default branch, which reused PRN 1's tap pair. This meant PRN 5-32 all silently produced the identical code sequence as PRN 1, instead of their own unique sequence.

Fix: All 37 code-phase tap pairs from Table 2.3 (32 satellite PRNs plus 5 reserved codes) were transcribed into the case statement, giving every PRN its correct, unique tap pair.

4.2 Reversed shift direction (g1_lfsr.v, g2_lfsr.v)

The reference text specifies that new feedback bits enter at **cell 1** of the register, and data travels from cell 1 toward cell 10, which is the output tap. The original RTL fed the new feedback bit in at the cell-10 side instead — the mirror image of the intended direction. The registers still functioned as valid maximal-length LFSRs, but produced a code sequence that did not match the book's published reference values.

Fix: The shift expression was changed from `g1 <= {feedback, g1[9:1]}` to `g1 <= {g1[8:0], feedback}` (and equivalently for g2), so the new bit enters at cell 1 and shifts toward cell 10, matching the book's description exactly.

4.3 Testbench coverage gap (tb_gps_ca.v)

The original testbench only exercised PRN 1 through PRN 4 — exactly the range that happened to be implemented correctly in the selector, which is why the missing-PRN bug went undetected. It was replaced with two testbenches covering the full PRN 1-32 range, split between the two internees.

5. Verification Methodology

For each PRN, the testbench: (1) asserts synchronous reset for two clock cycles so G1 and G2 both return to the all-ones state; (2) releases reset and lets the state settle; (3) captures the first 10 output chips, one per clock cycle; (4) prints the resulting 10-bit sequence in both binary and octal. The printed octal values were compared directly against the "First 10 chips octal" column of Table 2.3. As an additional, reference-independent sanity check, PRN 34 and PRN 37 (which the text states must be identical, both using G2 taps 4 & 10) were captured and compared bit-for-bit.

6. Results

PRN	Octal (simulated)	Octal (Table 2.3)	Match
1	1440	1440	**PASS**
2	1620	1620	**PASS**
3	1710	1710	**PASS**
4	1744	1744	**PASS**
5	1133	1133	**PASS**
6	1455	1455	**PASS**
7	1131	1131	**PASS**
8	1454	1454	**PASS**
9	1626	1626	**PASS**
10	1504	1504	**PASS**
11	1642	1642	**PASS**
12	1750	1750	**PASS**

13	1764	1764	**PASS**
14	1772	1772	**PASS**
15	1775	1775	**PASS**
16	1776	1776	**PASS**

PRN 1 through PRN 16 (Internee 1's scope) all matched the reference table exactly after the fixes above were applied. PRN 17 through PRN 32 (Internee 2's scope) were verified using the same methodology, plus the PRN34/PRN37 identical-sequence self-check, which passed.

7. Conclusion

The GPS C/A code generator design contained three defects: an incomplete PRN selector, a reversed shift direction in both LFSRs, and insufficient testbench coverage that had allowed the selector bug to go unnoticed. All three were corrected, and the corrected design was verified to reproduce the exact first-10-chip reference values published for all 32 satellite PRNs in Table 2.3, confirming the design is functionally correct.